GTBank SQL Project

Customer Transaction Analysis System

Client:	Guaranty Trust Bank (GTBank)
Position:	Junior Data Analyst
Project Duration:	2 Weeks
Difficulty Level:	■■■ Intermediate

Business Context

Guaranty Trust Bank (GTBank) operates 25 branches across the country, serving over 50,000 customers with various account types including savings, checking, and fixed deposits. The bank currently processes approximately 10,000 transactions daily through ATMs, online banking, and branch counters.

You have been hired as a **Junior Data Analyst** at GTBank. The bank's management team needs your expertise to build a comprehensive database system that will help them make data-driven decisions and improve customer service.

Business Challenges

The bank is currently facing several critical challenges:

1. Fraud Detection Gap

The bank lacks a systematic way to identify unusual transaction patterns. Recent incidents have shown customers making large withdrawals without proper verification flags. Management needs a system to automatically identify transactions exceeding $10,000 for review and monitoring purposes.

2. Customer Retention Issues

Premium customers are leaving for competitors. The bank doesn't have clear visibility into which customers are most valuable based on their account balances and transaction volumes. There's no way to identify and reward high-value customers or create targeted retention programs.

3. Branch Performance Blind Spots

Branch managers have no data-driven insights into which branches are underperforming or growing. The board wants to optimize branch operations based on customer activity and account distribution, but current reporting is manual and inconsistent.

4. Inactive Account Problem

Thousands of dormant accounts with low balances (less than $100) consume system resources. The bank needs to identify these accounts for closure or reactivation campaigns to reduce operational costs and improve efficiency.

5. Loan Cross-Selling Opportunity

The lending department wants to target existing high-value customers for loan products, but they don't have integrated data showing customer financial profiles. Customers with high balances and good transaction history are ideal loan candidates, but there's no system to identify them.

6. Compliance Requirements

Banking regulations require the bank to monitor and report transactions above $10,000 for anti-money laundering (AML) purposes. Currently, this is done manually, creating compliance risks and consuming staff time.

Your Task

The Head of Operations has assigned you to build a **Transaction Analytics Database** that will solve these challenges. Your database must enable the bank to:

1. Identify high-value customers who contribute most to the bank's deposits
2. Flag suspicious transactions for the fraud prevention team
3. Generate reports showing branch performance metrics
4. Identify dormant accounts and frequent transactors
5. Support the loan department in identifying potential loan customers
6. Provide data for regulatory compliance reporting
7. Track customer transaction patterns and account balances
8. Enable data-driven decision making across all departments

Database Design

You will design and implement a relational database with **5 interconnected tables** to track all aspects of the bank's operations:

Table 1: customers

Stores complete customer information including personal details and account tier classification.

Column Name	Data Type	Description	Constraints
customer_id	INT	Unique customer identifier	PRIMARY KEY
first_name	VARCHAR(50)	Customer's first name	NOT NULL
last_name	VARCHAR(50)	Customer's last name	NOT NULL
email	VARCHAR(100)	Customer's email address	UNIQUE, NOT NULL
phone	VARCHAR(20)	Phone number	
date_of_birth	DATE	Date of birth	
registration_date	DATE	When customer joined the bank	NOT NULL
customer_type	VARCHAR(20)	Savings, Current, or Premium	

Table 2: accounts

Tracks all bank accounts. Each customer can have multiple accounts of different types.

Column Name	Data Type	Description	Constraints
account_id	INT	Unique account identifier	PRIMARY KEY
customer_id	INT	Links to customers table	FOREIGN KEY, NOT NULL
account_number	VARCHAR(20)	Unique account number	UNIQUE, NOT NULL
account_type	VARCHAR(20)	Savings, Checking, Fixed Deposit	NOT NULL
balance	DECIMAL(15,2)	Current account balance	DEFAULT 0
opening_date	DATE	When account was opened	NOT NULL
branch_id	INT	Links to branches table	NOT NULL

status	VARCHAR(20)	Active, Frozen, or Closed	DEFAULT 'Active'

Table 3: transactions

Records every transaction made by customers. This is the heart of the analytics system.

Column Name	Data Type	Description	Constraints
transaction_id	INT	Unique transaction identifier	PRIMARY KEY
account_id	INT	Links to accounts table	FOREIGN KEY, NOT NULL
transaction_type	VARCHAR(20)	Deposit, Withdrawal, Transfer	NOT NULL
amount	DECIMAL(15,2)	Transaction amount	NOT NULL
transaction_date	DATETIME	When transaction occurred	NOT NULL
description	TEXT	Transaction description	
balance_after	DECIMAL(15,2)	Account balance after transaction	

Table 4: branches

Information about all bank branches for performance analysis.

Column Name	Data Type	Description	Constraints
branch_id	INT	Unique branch identifier	PRIMARY KEY
branch_name	VARCHAR(100)	Name of the branch	NOT NULL
city	VARCHAR(50)	Branch city	
state	VARCHAR(50)	Branch state	
manager_name	VARCHAR(100)	Branch manager's name	

Table 5: loans

Tracks customer loans for cross-selling analysis.

Column Name	Data Type	Description	Constraints
loan_id	INT	Unique loan identifier	PRIMARY KEY
customer_id	INT	Links to customers table	FOREIGN KEY, NOT NULL
loan_type	VARCHAR(30)	Personal, Home, Auto, Education	

loan_amount	DECIMAL(15,2)	Total loan amount	NOT NULL
interest_rate	DECIMAL(5,2)	Annual interest rate (%)	
loan_date	DATE	When loan was issued	NOT NULL
loan_status	VARCHAR(20)	Active, Paid, Defaulted	

Sample Data Requirements

You must insert realistic sample data to test your database. **Minimum requirements:**

Table	Minimum Rows	Notes
customers	30	Mix of Savings, Current, and Premium customers
accounts	45	Some customers should have multiple accounts
transactions	150	Covering at least 3 months of activity
branches	5	Different cities/states
loans	20	Mix of loan types and statuses

Data Quality Tips:

- Use realistic names, email addresses, and phone numbers
- Ensure account balances are realistic (not all customers should have $1 million)
- Create varied transaction amounts (small daily transactions and large occasional ones)
- Include some transactions over $10,000 for fraud detection testing
- Make sure some customers have multiple accounts at different branches
- Include both active and dormant accounts (low balance, no recent transactions)

Required SQL Queries

Write the following **15 SQL queries** to answer critical business questions. Each query must be properly formatted with comments explaining what it does.

Query 1: Find all customers who have more than one account.

Category: Customer Intelligence

Hint: Use GROUP BY with HAVING COUNT(*) > 1

Query 2: Calculate the total balance across all accounts for each customer. Show customer name and total balance.

Category: Customer Intelligence

Hint: Join customers and accounts, use SUM() with GROUP BY

Query 3: Find the top 5 customers with the highest total transaction amounts (sum of all deposits and withdrawals).

Category: Customer Intelligence

Hint: Join customers, accounts, and transactions, use SUM() and ORDER BY with LIMIT

Query 4: List all transactions greater than $10,000. Show customer name, transaction amount, date, and type.

Category: Fraud & Compliance

Hint: Join transactions, accounts, and customers with WHERE amount > 10000

Query 5: Calculate the average account balance by account type (Savings, Checking, Fixed Deposit).

Category: Analytics

Hint: Use AVG(balance) with GROUP BY account_type

Query 6: Find customers who have made more than 10 withdrawals in the last month.

Category: Fraud Detection

Hint: Filter by transaction_type and date, use GROUP BY with HAVING

Query 7: List all branches and the total number of accounts in each branch. Sort by highest number first.

Category: Branch Performance

Hint: Join branches and accounts, use COUNT() with GROUP BY and ORDER BY DESC

Query 8: Find customers who have both a loan and a savings account.

Category: Cross-Selling

Hint: Use subqueries or JOIN with conditions on account_type and loan existence

Query 9: Calculate the total loan amount by loan type and loan status.

Category: Loan Analysis

Hint: Use SUM(loan_amount) with GROUP BY loan_type, loan_status

Query 10: Find accounts with balance less than $100 (low balance accounts).

Category: Account Maintenance

Hint: Simple WHERE balance < 100 on accounts table

Query 11: Use a subquery to find customers whose total account balance is greater than the average balance of all customers.

Category: Advanced Analytics

Hint: Subquery: SELECT AVG(balance) FROM accounts, compare in main query

Query 12: Find the most active account (account with the most transactions).

Category: Customer Behavior

Hint: Join accounts and transactions, use COUNT() with GROUP BY and ORDER BY DESC LIMIT 1

Query 13: Calculate monthly transaction volume (total amount transacted each month).

Category: Revenue Analysis

Hint: Use DATE functions to extract month, SUM(amount), GROUP BY month

Query 14: Find customers who have never made a transaction.

Category: Inactive Customers

Hint: Use LEFT JOIN or NOT IN with subquery

Query 15: Generate a customer statement: Show customer name, account number, and all their transactions for a specific customer (e.g., customer_id = 5).

Category: Customer Service

Hint: Join all three tables with WHERE customer_id = 5

Project Deliverables

You must submit the following files:

1. create_tables.sql

Contains all CREATE TABLE statements for the 5 tables with proper data types, primary keys, and foreign keys.

2. insert_data.sql

Contains all INSERT statements with the minimum required sample data (30 customers, 45 accounts, 150 transactions, 5 branches, 20 loans).

3. queries.sql

Contains all 15 required queries with clear comments explaining each query's purpose.

4. project_report.pdf

A 2-3 page document containing:
- Database schema diagram (can be hand-drawn and scanned)
- Query results (screenshots or formatted text output)
- Key findings and insights from your analysis
- At least 3 business recommendations based on your findings

Grading Rubric (100 Points)

Component	Points	Criteria
Database Design	15	All 5 tables created correctly with proper keys and relationships
Sample Data Quality	10	Realistic data, meets minimum requirements, variety of scenarios
Basic Queries (1-8)	20	Correct syntax, produces expected results
Complex Queries (9-15)	25	Proper use of subqueries, JOINs, aggregates, and advanced concept
Query Documentation	20	Clear comments, well-formatted code, results documented
Project Report	10	Professional presentation, meaningful insights, business recommenda

Note: Queries must run without errors and produce meaningful results. Syntax errors or queries that don't answer the business question will receive partial credit only.

Tips for Success

1. **Start with ERD:** Before writing any SQL, sketch the relationships between tables. Understand which tables connect to which through foreign keys.

2. **Test Incrementally:** Don't write all tables at once. Create one table, insert data, test it, then move to the next.

3. **Use Realistic Data:** Don't use 'Test Customer 1', 'Test Customer 2'. Use real-sounding names like 'John Smith', 'Sarah Johnson'. This makes testing more meaningful.

4. **Check Foreign Keys:** Before inserting into accounts table, make sure the customer_id exists in customers table. Before inserting transactions, make sure account_id exists.

5. **Comment Your Code:** Every query should have a comment explaining what business question it answers.

6. **Verify Results:** After each query, ask yourself: 'Does this answer make sense?' If you have 30 customers and a query returns 100, something is wrong.

7. **Use Proper Formatting:** Indent your SQL code properly. Use uppercase for SQL keywords (SELECT, FROM, WHERE) and lowercase for table/column names.

8. **Handle Edge Cases:** What if a customer has no transactions? What if an account has zero balance? Your queries should handle these gracefully.

9. **Document Assumptions:** In your report, explain any assumptions you made (e.g., 'I assumed transactions are in USD', 'I assumed one customer can have max 5 accounts').

10. **Proofread:** Before submission, run every query one more time to ensure it works. Check for typos in table names, column names, etc.

Real-World Impact

This project simulates a real data analyst role at a bank. The skills you develop here are directly applicable to actual jobs in:

- **Banking & Finance:** Transaction analysis, fraud detection, customer segmentation
- **E-Commerce:** Sales analytics, customer behavior analysis, inventory management
- **Healthcare:** Patient analytics, appointment scheduling, billing analysis
- **Retail:** Product performance, customer loyalty programs, sales forecasting
- **Technology:** User behavior analysis, subscription management, usage patterns

Upon completing this project, you will be able to demonstrate proficiency in:

✓ Database design and normalization
✓ Writing complex SQL queries with multiple JOINs
✓ Using aggregate functions and GROUP BY for analysis
✓ Implementing subqueries for advanced filtering
✓ Translating business questions into SQL queries
✓ Presenting data insights to non-technical stakeholders

Submission Guidelines

Deadline: 2 weeks from assignment date

Submission Format:

Create a folder named **YourName_GTBank_Project** containing:

- create_tables.sql
- insert_data.sql
- queries.sql
- project_report.pdf
- README.txt (optional: any notes or special instructions)

Compress the folder into a ZIP file and submit via the learning management system.

Academic Integrity:

This is an individual project. You may discuss concepts with classmates, but all code and analysis must be your own work. Copying code from others or the internet (beyond syntax references) constitutes plagiarism and will result in zero credit.

Recommended Resources

SQL Syntax References:

- W3Schools SQL Tutorial: https://www.w3schools.com/sql/
- SQL Cheat Sheet: Google 'SQL cheat sheet' for quick syntax reference
- Your class notes on JOINs, GROUP BY, and subqueries

Sample Data Generation:

- Mockaroo (mockaroo.com): Generate realistic sample data
- Faker libraries: For programmatic data generation
- Manual creation: Sometimes writing INSERT statements manually gives you better control

Database Tools:

- MySQL Workbench: Visual database design and query execution
- phpMyAdmin: Web-based database management
- SQLite Browser: Lightweight option for local development
- DBeaver: Universal database tool supporting multiple DBMS

Good luck with your project!

Remember: Data analysts solve business problems with data. Your job isn't just to write SQL - it's to find insights that help GTBank serve customers better and operate more efficiently.